Clase 21 — Control de congestión en TCP: slow start, AIMD, fast recovery y ECN

Bloque: C (Transporte) · **Semana:** 7 · **Duración estimada:** 110 min **Correlativa:** Clase 20 (control de flujo vs. de congestión) · **Sigue con:** Clase 22 (taller TCP) **Material de referencia:** Tanenbaum cap. 6 (§6.7) · RFC 9293, RFC 5681, RFC 9438, RFC 3168, RFC 9768, RFC 9331, RFC 6928

🎯 Al terminar esta clase vas a poder

1. Explicar por qué TCP necesita un mecanismo de control de congestión **separado** del control de flujo, y por qué el control de flujo no alcanza.
2. Escribir la **ventana efectiva** de un segmento TCP como el mínimo entre `cwnd` y `rwnd`, y justificar por qué es un mínimo y no una suma.
3. Recitar los **cuatro algoritmos** del control de congestión de TCP (Tahoe) y ejecutar la evolución de `cwnd` durante una conexión completa, RTT por RTT.
4. Explicar por qué el **timeout** y los **3 ACKs duplicados** se tratan distinto, y qué hace cada uno.
5. Describir el mecanismo de **ECN**: qué son los 2 bits del campo ECN, qué hace cada router y qué hace cada extremo.
6. **Implementar en pseudocode** la detección y el control de congestión, incluyendo ECN, respetando las primitivas que da el enunciado.

> **Prerequisito del examen:** `2011-03 Problema 1` (30 puntos — el problema de ECN), `2008-07 P2` (10 pts), `2009-07 P1` (10 pts), `2009-12 P5`, `2008-12 P2`, `2010-02 Problema 2`.

PARTE 1 — TEORÍA

1.1 El problema que TCP tiene que resolver

Imaginá que activás una descarga. Tu disco de 500 Mbps no te sirve de nada si el enlace de la casa te da 10 Mbps. Y al revés: un enlace de 100 Mbps que te comparte con 40 personas, los otros 39, tu descarga de 40 Mbps te va a parecer lenta.

Hay **dos** problemas distintos, y no son el mismo:

```
┌─ El emisor manda MÁS RÁPIDO de lo que el receptor puede absorber ─┐
│ → CONTROL DE FLUJO │
│ (lo arregla rwnd, clase anterior) │
└──────────────────────────────────────────────────────────────────┘

┌─ La RED en el medio no tiene recursos para lo que todos mandan ────┐
│ → CONTROL DE CONGESTIÓN │
│ (lo arregla cwnd, esta clase) │
└──────────────────────────────────────────────────────────────────┘
```

La diferencia de fondo:

	Control de flujo	Control de congestión
El problema es de	un host lento	**toda la red**
El síntoma es	el buffer del receptor se llena	los buffers de **los routers** se llenan
La pérdida significa	"no leíste rápido"	"**la red está saturada**"
La ventana la controla	el receptor (rwnd)	el emisor, estimando (cwnd)
Se arregla con	ACK diferidos / zero-window probe	**reducir el ritmo de envío**

Y acá está el punto clave para el examen: TCP **no puede distinguir** una pérdida por buffer lleno del receptor (control de flujo) de una pérdida en la red (congestión). Para TCP, un paquete perdido es un paquete perdido. Por eso el mismo mecanismo (disminuir la ventana) sirve para las dos cosas, y por eso la "interpretación" que TCP hace es conservadora.

La evidencia de que la red se congestiona

TCP detecta tres señales, en orden creciente de gravedad:

1. **No hay pérdida, pero el RTT subió mucho** → alguien está encolando. No es un evento, TCP no reacciona formalmente.
2. **3 ACKs duplicados** → hubo una pérdida, pero el resto de la ventana llegó igual. Mildeprioridad: pérdida aislada.
3. **Timeout (RTO expirado)** → la pérdida es *grave*: no llegó ni siquiera el ACK del siguiente. La ventana entera está frenada.

Esta distinción es la que te van a preguntar, y es la clave del problema de 2011.

1.2 Las dos ventanas y la ventana efectiva

El emisor TCP manda como máximo esta cantidad de bytes sin esperar ACK:

```
bytes en vuelo ≤ min(cwnd, rwnd)
               ─────────  ────
               congestión flujo
```

- `cwnd` **(congestion window):** la **hipótesis del emisor** sobre cuánta congestión hay. Empieza en un valor pequeño y la va ajustando.
- `rwnd` **(receive window):** lo que el receptor dice que le entra en su buffer. Va en el campo *window* de cada segmento TCP que manda.

¿Por qué el mínimo y no la suma? Porque **el emisor no puede mandar más de lo que la red aguanta** (cwnd), ni **más de lo que el receptor aguanta** (rwnd). Ambas son restricciones *superiores*. Si las sumaras, un solo paquete perdido por congestión en la red se acumularía en el buffer del receptor y lo desbordaría justamente cuando la red va peor — que es cuando menos conviene. El `min` es el que garantiza que **nadie se desborda**: ni la red (por cwnd), ni el receptor (por rwnd).

En los exámenes los dos los llaman **"ventana efectiva"** o **"ventana de congestión"** según el caso. Si el enunciado habla de una sola ventana, asumí que `rwnd` es grande y el límite es `cwnd`.

1.3 Las tres variables de estado

```
cwnd = ventana de congestión actual, en bytes. Empieza en un MSS (o un
Initial Window del RFC 6928, ver Actualización).
ssthresh = "slow start threshold", el umbral a partir del cual se pasa de
slow start a congestion avoidance. Empieza muy alto.
dupACK = contador de ACKs duplicados quejetsnorth se recibieron seguidos.
```

`ssthresh` es la variable que decide la regla:

> Si `cwnd < ssthresh` → **slow start**. Si `cwnd >= ssthresh` → **congestion avoidance**.

La pérdida no es lo único que cambia de régimen: cuando `cwnd` *crece* y supera el umbral, también se cambia de régimen.

1.4 Los cuatro algoritmos (Tahoe, 1988)

(a) Slow start — "arranque lento"

No es lento en la velocidad: es lento en **el crecimiento**.

```
Por cada ACK recibido:
cwnd ← cwnd + 1 MSS (por ACK, no por RTT)
```

O sea: **cada segmento confirmado duplica la ventana**:

```
ACK de 1 MSS recibido → cwnd = 1 + 1 = 2 MSS
ACK de 1 MSS recibido → cwnd = 2 + 1 = 3 MSS
...
```

Después de un RTT completo:

```
RTT 1: cwnd = 1 MSS (mandás 1 MSS, lo confirmás)
RTT 2: cwnd = 2 MSS (mandás 2, los 2 ACKs te suman 2)
RTT 3: cwnd = 4 MSS (mandás 4, los 4 ACKs te suman 4)
RTT 4: cwnd = 8 MSS
...
```

Es decir **exponencial en base 2**: `cwnd(n) = 2^(n−1) · MSS`. Por eso se llama "arranque lento": en una red con RTT de 100 ms pasás de 1 MSS a 64 KB en 7 RTT, o sea en 0,7 segundos.

¿Por qué "slow" y no "start"? Porque en la versión clásica el emisor arrancaba mandando a toda velocidad y el primer pico de congestión le costaba la mitad de la ventana. Empezar en 1 MSS es "perezosamente" aceptar que no conoce la red.

(b) Congestion avoidance — "evitación de la congestión"

Se entra cuando `cwnd >= ssthresh`. Ahora el crecimiento es **lineal**:

```
Por cada ACK recibido:
cwnd ← cwnd + MSS · MSS / cwnd (un MSS por RTT)
```

Escribido así parece raro, pero es equivalente a la forma más conocida:

```
cwnd ← cwnd + MSS por cada RTT
```

La forma `MSS²/cwnd` es la que aparece **literalmente** en la solución oficial del examen de 2011-03, y es la que te van a pedir. La justification: cada ACK confirma `MSS` bytes, así que el emisor puede mandar `MSS` bytes más; repartido eso entre los `cwnd/MSS` ACKs que llegan en el RTT, da `MSS / (cwnd/MSS) = MSS²/cwnd` por ACK. Sumado a lo largo de un RTT, `cwnd/MSS · MSS²/cwnd = MSS`.

(c) Fast retransmit

Cuando llegan **3 ACKs duplicados**, no esperás al timeout:

```
3 ACKs duplicados → retransmitir el segmento perdido YA,
sin esperar a que venza el RTO
```

¿Por qué 3 y no 1? Porque con ACKs **acumulativos**, un ACK duplicado puede significar dos cosas:

- El segmento que esperaba **realmente** se perdió (pérdida), o
- Simplemente el ACK se está **reordenando** en la red y el que "falta" llegó después (reordenamiento).

Con 3 duplicados es estadísticamente muy improbable que sea reordenamiento, así que se asume pérdida. **Esto es un compromiso entre latencia y false positives**, y es exactamente lo que te preguntan en `2008-07 P2`.

(d) Fast recovery

El refinamiento: con 3 dupACKs **no se tira la ventana a 1 MSS**, porque los datos que vienen detrás son la señal de que la red **aguanta** ese ritmo.

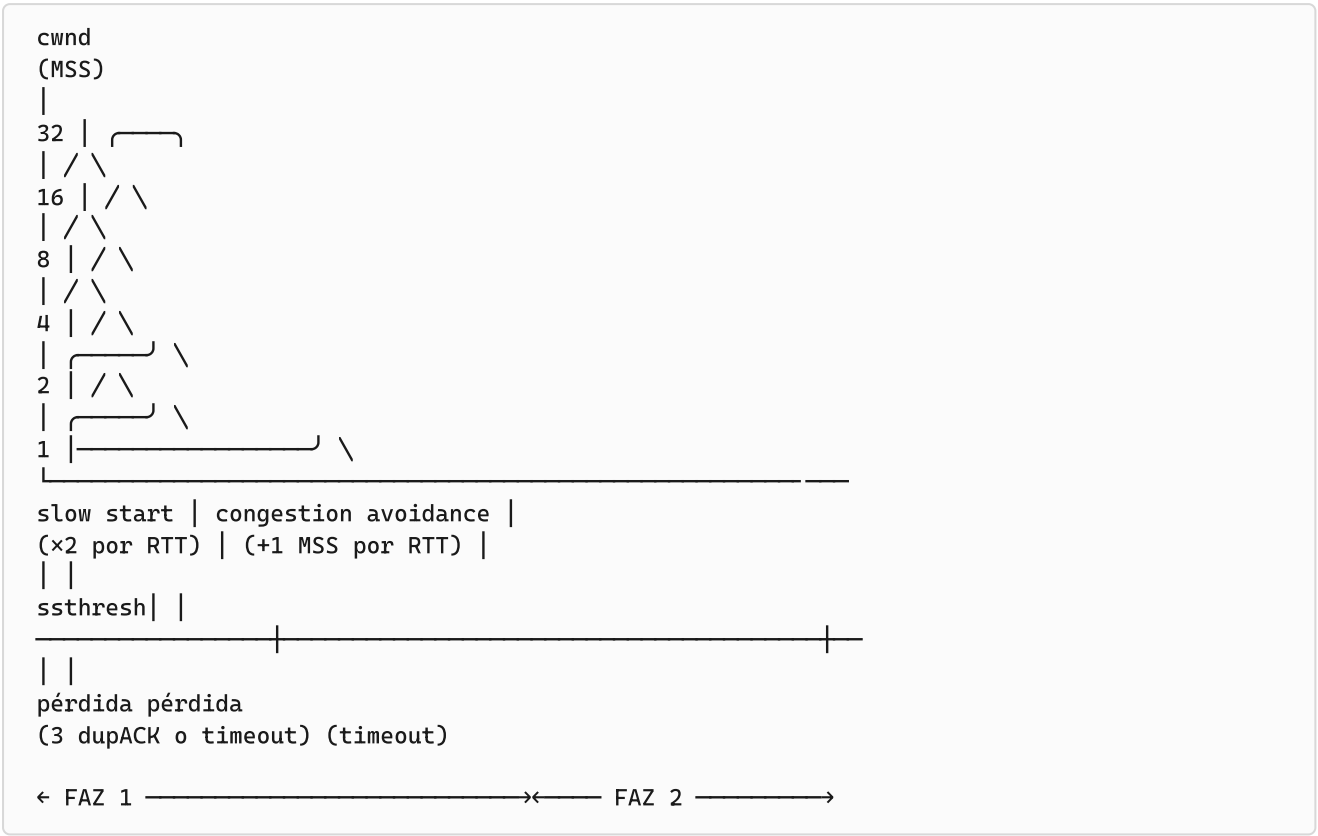
```
3 dupACKs → cwnd = ssthresh (se parte por la mitad, no a 1 MSS)
ssthresh = cwnd / 2
retransmitir el perdido
→ seguir acknowledging, sin cortar la ventana a 1 MSS
Timeout → cwnd = 1 MSS (ESTO sí es Tahoe puro)
ssthresh = cwnd_antes / 2
→ slow start
```

La asimetría es el punto clave y lo que más se confunde:

```
| 3 dupACKs | Timeout
──────────┼──────────────┼──────────────
Interpretación|red:tensa |red:muerta
cwnd nuevo | cwnd/2 | 1 MSS
Régimen | (ev. cong.) | slow start
Latencia | baja (inmediata) | alta (espera RTO)
```

Tahoe es el que hace `cwnd = 1 MSS` en los dos casos. **Reno** (1989) es el que distingue. Los exámenes de FING por lo general describen **Tahoe** (cuatro algoritmos, `cwnd = 1 MSS` ante cualquier pérdida) o la distinción, y en la solución de `2011-03` te piden slow start para los dos casos. Leelo bien.

1.5 Diagrama de la evolución de `cwnd`



Leelo así: **el doble en la primera fase, la recta en la segunda, y el salto a cero en cada pérdida**. Ese gráfico es el 80% de la respuesta a `2008-07 P2`.

1.6 ECN — Explicit Congestion Notification

El problema que resuelve

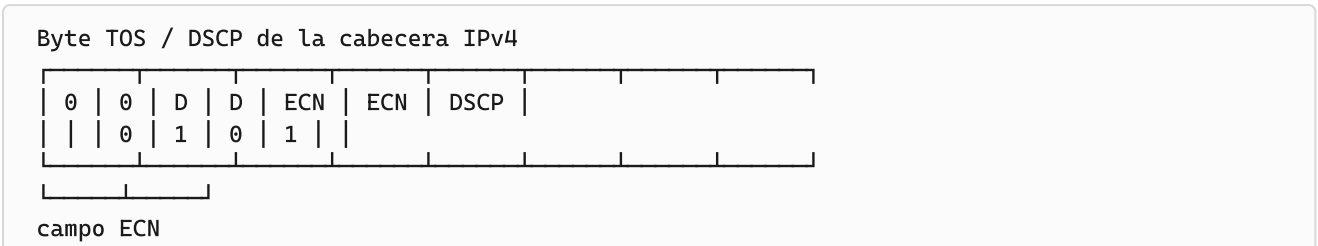
La detección de congestión de TCP es **reactiva**: alguien tiene que perder un paquete para que TCP se entere. Eso significa:

1. **Se pierde un paquete que no hacía falta perder.** Ese paquete se iba a descartar igual, así que no es una pérdida "de verdad"… pero para la aplicación que lo esperaba, sí lo es.
2. **Los routers intermedios no participan.** TCP deduce congestión de una pérdida, pero el que sabe que está congestionado es el **router** que threwó el paquete. Y no se lo dice.

La solución

ECN (RFC 3168, 2001): que el router **no descarte** el paquete sino que le **ponga una marca**, y que TCP reaccione a la marca en lugar de a la pérdida.

Hay **2 bits** en la cabecera IP, en el byte TOS/DSCP:

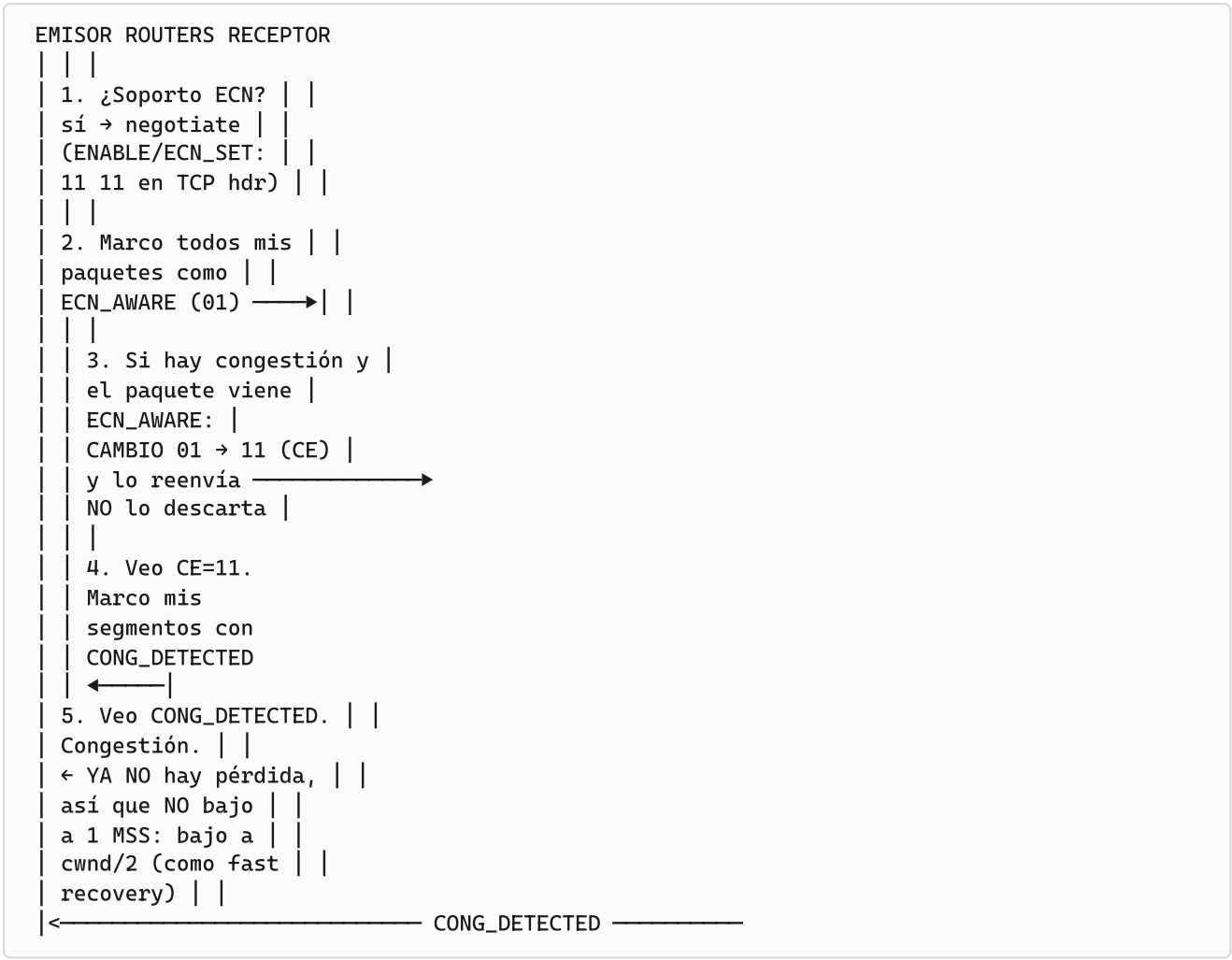


Los 2 bits del campo ECN (antes DSCP) toman 3 valores:

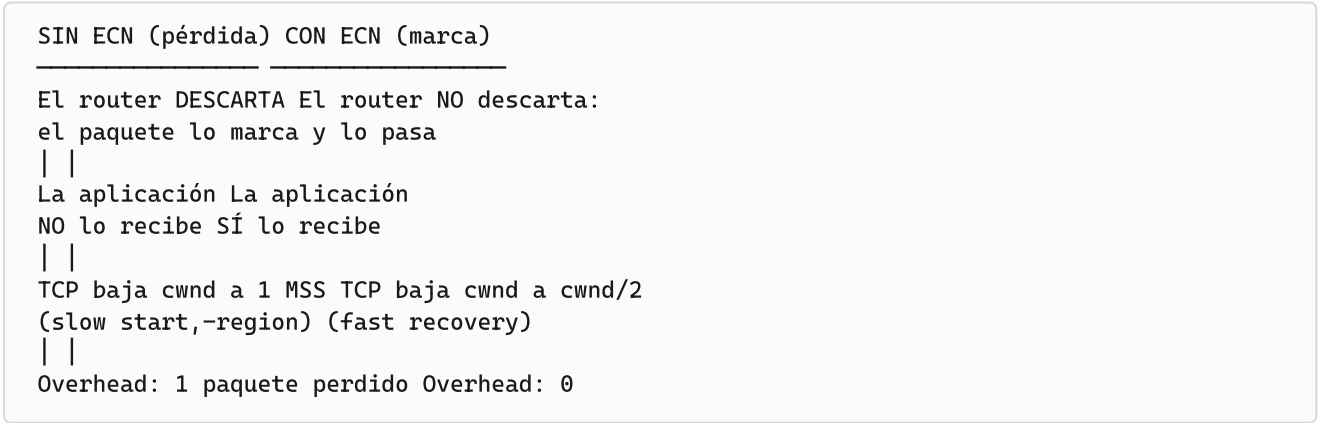
Valor	Bits	Significado
`00`	0-0	**NO_ECN** — el origen no sabe usar ECN
`01`	0-1	**ECN_AWARE** — "sé usar ECN, pero ahora no hay congestión"
`11`	1-1	**ECN_CONG** (= **CE**, *Congestion Experienced*) — "**hubo congestión**"
`10`	1-0	**CE**, pero originated por un router que no entiende ECN (legado)

> ⚠️ **Ojo:** la combinación `10` es válida **solo** en el paquete que viaja en la **dirección de vuelta**. `10` no lo puede poner un origen, porque implicaría que un host entendió ECN y sin embargo no negoció ECN Setup. Es un detalle sutil que aparece en la RFC y que un profesordetail-oriented puede preguntar.

El protocolo, extremo por extremo



ECN vs. pérdida: la diferencia clave



Eso es ECN en su forma más pura: **cero paquetes perdidos a cambio de un bit**.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - Los **2 bits** del campo ECN y sus 3 (4) valores `00/01/11/10`. - La secuencia de **4 pasos**: emisor marca ECN_AWARE → router cambia a CE → receptor marca

CONG_DETECTED → emisor reduce cwnd. - La idea de que **ECN evita la pérdida** y por eso permite una reducción de ventana **menos agresiva**. - `CE` = *Congestion Experienced* = `ECN_CONG`.

Cambió o se modificó — y acá hay que ser preciso, porque es donde más se matan las clases:

- `RFC 7560` **NO es ECN++.** `RFC 7560` (2010, *Informational*) es solo el **documento de problema y requisitos** de "ECN más preciso". No especifica ningún mecanismo. Mucha gente lo cita mal como si fuera la especificación.
- `RFC 8951` **NO tiene nada que ver con ECN.** Es una aclaración de **EST (Enrollment over Secure Transport)**. No la invoques nunca en este contexto.
- **La implementación real de "ECN++" quedó en** `RFC 9768 – AccECN` (*More Accurate Explicit Congestion Notification Feedback in TCP*), que además **actualiza** la RFC 3168. *Verificá el número en* `rfc-editor.org/info/rfc9768` *antes de citarlo en un examen: es de 2026 y podría haber cambiado.*
- **La reacción ante un CE ya no es "bajar a 1 MSS".** La especificación moderna pide una reducción **proporcional y menos agresiva** — esencialmente la misma que ante fast recovery (`cwnd/2`) — porque el paquete **no se perdió**. Ese argumento conceptual es lo que hay que saber; el número exacto, menos.
- **L4S (`RFC 9331`, 2023)** es el paso siguiente: en vez de esperar a llenarse el buffer, el router marca **con un umbral mucho más bajo**, lo que da latencia muy baja sin necesitar descartes. Va de la mano con AQMs como **CoDel**.
- **`ABE` (`RFC 8511`, 2018)** — *Alternative Backoff with ECN* — cómo hacer el backoff cuando el receptor también sufrió pérdida por el CE.
- **Despliegue real:** ECN se **negocia**, así que solo se usa si **ambos** extremos lo soportan. Está activo por defecto en Linux (desde el kernel 4.1) y en Windows 10. **No es obligatorio**: no te lo pregunten como si lo fuera.

1.7 ¿Y el Initial Window?

La RFC 5681 decía "arrancá con 1 MSS". En la práctica todos arrancaban con distintos valores (Solaris 8 KB, Linux 16 KB…), lo que rompía la independencia de la red.

RFC 6928 (2013) — Increasing TCP's Initial Window (IW): estandariza un IW de **10 segmentos (IW = 10 · MSS)**. Es el valor por defecto en Linux, Windows y macOS hoy.

> **Consecuencia para el examen:** si el enunciado dice "slow start empieza en cwnd = 1 MSS", usá 1 MSS (es lo que dicen los apuntes y las soluciones oficiales de 2011). Si dice "IW por defecto actual", es 10 MSS (RFC 6928). **Citá las dos cosas** y seguí lo que pida el enunciado: lo que se corrige es lo que dice el examen.

1.8 Una mención a TCP Vegas y a lo que se usa hoy

- **TCP Vegas (1996):** no espera a la pérdida. Mide el RTT y lo compara con el RTT mínimo teórico (`RTT_min + L/R`). Si la diferencia crece, hay cola → baja cwnd. **Ventana de congestiónpredictiva**, sin

pérdidas.

- **BBR (Google):** modelo de **bottleneck bandwidth** y RTT mínimo, no de ventana. Mucho mejor en rutas con pérdida no relacionadas con congestión (WiFi, buffers intermedios). Implementado en Linux, en draft de IETF (no RFC todavía).
- **CUBIC (RFC 9438, 2023):** el default en Linux, Windows y Apple. **No fue reemplazado** — lo que pasó es que pasó de *Informational* (RFC 8312, 2018) a *Standards Track*. A diferencia de AIMD, en lugar de sumar 1 MSS por RTT usa una curva cúbica que **apunta al último punto donde hubo pérdida**, lo que permite "resetear" la info de la red mucho más rápido en enlaces con BDP grande.

⚠️ **Para el examen:** el "AIMD" y los "cuatro algoritmos" **siguen siendo lo que se enseña y lo que se pregunta**, porque TCP Reno/Tahoe es la base de todos los demás. Si el enunciado te pide "el mecanismo de control de congestión de TCP", respondé slow start / congestion avoidance / fast retransmit / fast recovery, y si hay lugar, una línea de "hoy se usa CUBIC o BBR, pero la lógica es la misma".

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La curva completa de `cwnd` (Tahoe, MSS = 1 KB)

Datos. MSS = 1000 bytes. `cwnd` inicial = 1 MSS = 1000 B. `ssthresh` inicial = 64 KB = 64 MSS. En el RTT 5 se pierde un paquete y se detectan 3 ACKs duplicados.

a) Calculá `cwnd` al final de cada RTT del RTT 1 al 4.

Razonamiento. En slow start el crecimiento es `cwnd = 2 · cwnd_anterior` por RTT. La clave es **por qué**: en el RTT *n* mandás `cwnd(n−1)` bytes; llegan esos tantos ACKs; cada ACK suma 1 MSS; entonces `cwnd(n) = cwnd(n−1) + cwnd(n−1) = 2·cwnd(n−1)`. O sea, **el número de ACKs que recibo en un RTT es igual a la ventana que tenía**.

```
RTT 1: cwnd_enviado = 1 MSS → llega 1 ACK → cwnd = 1 + 1 = 2 MSS
RTT 2: cwnd_enviado = 2 MSS → llegan 2 ACK → cwnd = 2 + 2 = 4 MSS
RTT 3: cwnd_enviado = 4 MSS → llegan 4 ACK → cwnd = 4 + 4 = 8 MSS
RTT 4: cwnd_enviado = 8 MSS → llegan 8 ACK → cwnd = 8 + 8 = 16 MSS
```

RTT	cwndbeginning	ACKs recibidos	cwnd al final
1	1 MSS	1	**2 MSS** = 2 000 B
2	2 MSS	2	**4 MSS** = 4 000 B
3	4 MSS	4	**8 MSS** = 8 000 B
4	8 MSS	8	**16 MSS** = 16 000 B

b) ¿En qué RTT se deja de estar en slow start?

Razonamiento. La regla es `cwnd < ssthresh` → slow start. Con `ssthresh = 64 MSS`: el slow start termina cuando `cwnd` llega a 64. Desde 16 hay que seguir duplicando: 32, luego 64. Como el umbral se alcanza al final del RTT 6, y la comparación es `cwnd >= ssthresh`, **el RTT 7 ya corre en congestion avoidance**.

```
RTT 5: 32 MSS (< 64 → slow start)
RTT 6: 64 MSS (== 64 → a partir de acá es congestion avoidance)
RTT 7: 65 MSS (+1 por RTT)
```

corríjanme si lo leés distinto: algunos apuntes dicen "termina en el RTT 6" (dejando de crecer cuando llega al umbral) y otros "en el RTT 7" (cuando el crecimiento pasa a ser lineal). **Lo que se corrige es el mecanismo**, y el mecanismo es la comparación `cwnd` vs. `ssthresh`. Decí las dos cosas.

c) En el RTT 5 hay 3 ACKs duplicados. ¿Qué pasa?

```
ANTES: cwnd = 32 MSS = 32 000 B
ssthresh = 64 MSS

Detección: 3 ACK duplicados → fast retransmit

Tahoe → ssthresh = cwnd/2 = 16 MSS
cwnd = 1 MSS
→ slow start de nuevo

Reno → ssthresh = cwnd/2 = 16 MSS
cwnd = ssthresh = 16 MSS
→ fast recovery
```

Interpretación. Con 3 dupACKs, Tahoe tira 32 KB de ventana a 1 KB. Eso es **castigar a la red** por una pérdida que ella no habríaisoftido si los buffers fueran más grandes. Reno dice: "los 32 KB igual circularon, o sea que la red los aguantó; bajemos a la mitad, no a cero".

d) Calculá `cwnd` en los 5 RTT siguientes, con Reno.

```
RTO/ACK : cwnd = 16 → 17 → 18 → 19 → 20 → 21 (congestion avoidance,
+1 MSS por RTT)
```

Y **con Tahoe**, en slow start desde 1:

```
RTT 1: 1→2 RTT 2: 2→4 RTT 3: 4→8 RTT 4: 8→16 RTT 5: 16→17 (¡superó ssthresh!)
```

2.2 Ejercicio 2 — La ventana efectiva

Datos. `cwnd = 8 KB`, `rwnd` anunciado por el receptor = 4 KB. RTT = 50 ms.

a) ¿Cuántos bytes puede mandar como máximo?

Razonamiento. Las dos son cotas superiores. La red admite 8 KB, el receptor 4 KB. Mandar 8 KB haría que 4 KB se acumulen en el buffer del receptor, que es justo lo que el rwnd existe para evitar.

```
ventana efectiva = min(cwnd, rwnd) = min(8 KB, 4 KB) = 4 KB
```

b) ¿Cuál es el throughput máximo?

Razonamiento. El throughput no es "bytes / RTT" sin más: es **cuántos bytes salen por segundo**. Si mando 4 KB cada 50 ms:

```
throughput = 4 KB / 0,050 s = 4000 / 0,050 = 80 000 B/s = 80 KB/s
= 640 kbps
```

c) Si el receptor no lee durante 1 segundo, ¿qué pasa con `rwnd`?

Razonamiento. El receptor no manda ACK, entonces el emisor no avanza la ventana de bytes confirmados. Cuando `rwnd` llega a 0:

```
R0 receptor anuncia rwnd = 20 KB, manda 20 KB
R1 sin ACK → el emisor ve rwnd = 20, ventana efectiva = min(cwnd, 20) = 20
R2 sin ACK → rwnd = 0 → ventana efectiva = 0 → NO PUEDE ENVIAR
```

TCP entra en **persistencia** (RFC 9293 §8.6) y manda **zero-window probes** (1 byte, con periodo exponencial) para preguntar "¿podés volver a leer?". Cuando el receptor vuelve a leer, manda un ACK con un `rwnd` nuevo y se reanuda.

> ⚠️ **Esto NO es control de congestión.** Es control de flujo, y es importantísimo no confundirlos: acá la red está perfecta; lo que está lento es la aplicación del otro lado. Si TCP bajara cwnd por esto, estaríamos castigando a la red por un problema del receptor. **Por eso son mecanismos separados.**

2.3 Ejercicio 3 — ECN en números

Datos. `cwnd = 20 MSS`, `ssthresh = 30 MSS`. Un router marca CE=11 un paquete.

a) ¿Qué haría TCP sin ECN? Tendría que **perder** ese paquete. 3 dupACK → Reno: `cwnd = 20/2 = 10`, `ssthresh = 10`. Tahoe: `cwnd = 1`.

b) ¿Qué hace con ECN (ECN++)?

```
Con ECN++ → cwnd = (20 − 1)/2 ≈ 9,5 ... pero lo que se usa es la misma
fórmula de fast recovery: cwnd = cwnd/2 = 10 MSS
ssthresh = 10 MSS
```

c) La diferencia, en números:

	Sin ECN	Con ECN
Paquetes perdidos	1	**0**
`cwnd` después	10 MSS (Reno)	10 MSS
`ssthresh` después	10 MSS	10 MSS

Interpretación — y acá está la moraleja. El mecanismo de reacción de TCP es **el mismo**. Lo que ECN cambia no es cuánto baja la ventana: es que **el paquete no se pierde**. Es un ejemplo perfecto de que en redes la pérdida de un paquete es un phenomenon diferente del phenomenon de congestión, aunque TCP las confunda.

2.4 Ejercicio 4 — Tipo examen nuevo: control de congestión con RTT variable

Enunciado. Un cliente TCP tiene `cwnd = 12 KB`, `ssthresh = 20 KB`, RTT inicial de 100 ms. En cada RTT manda `cwnd` bytes. En el RTT 4, un paquete se pierde y se detectan 3 ACKs duplicados. En el RTT 5 se detecta una pérdida por timeout.

a) `cwnd` y `ssthresh` al final de cada RTT, aplicando Tahoe.

RTT	`cwnd` inicio	Régimen	`cwnd` fin	`ssthresh`
1	12 KB	slow start (`12<20`)	24 KB	20 KB
2	24 KB	cong. avoid. (`24≥20`)	25 KB	20 KB
3	25 KB	cong. avoid.	26 KB	20 KB
4	26 KB	3 dupACK → pérdida	**1 KB**	**13 KB**
5	1 KB	timeout → pérdida	**1 KB**	**0,5 KB**

Justificación del RTT 4: `ssthresh = cwnd/2 = 26/2 = 13 KB`; `cwnd = 1 KB`. *Justificación del RTT 5:* timeout ⇒ Tahoe también va a 1 MSS, y `ssthresh = 1/2 = 0,5 KB`.

b) ¿Qué cambiaría con Reno?

```
RTT 4 (3 dupACK): ssthresh = 13 KB, cwnd = 13 KB (fast recovery)
RTT 5 (timeout): ssthresh = 0,5 KB, cwnd = 1 KB (slow start, igual que Tahoe)
```

c) En el RTT 2 el RTT subió de 100 ms a 180 ms. ¿Qué indica eso y qué debería hacer un diseño moderno?

Respuesta. Indica **cola** en los buffers de los routers: el paquete tardó más porque **estuvo esperando su turno**. El paquete **no se perdió**, así que los algoritmos basados en pérdida (Tahoe, Reno, CUBIC) **no reaccionan**: sigue creciendo `cwnd`.

Esto es un problema conocido como **bufferbloat**. Las soluciones modernas son:

- **ECN**: el router marca en vez de tirar (o hacer cola). Reacciona **antes** de la pérdida.
- **AQM (*Active Queue Management*)**: el router no espera a llenarse el buffer, marca o descarta temprano. **CoDel**, **FQ-CoDel** (por defecto en Linux desde 3.5), **PIE**. Unaim: "Bufferbloat — Better Bufferbloat" (RFC 9000).
- **AccECN**: la reacción menos agresiva.

> ⚠️ **Actualización a 2026:** el **bufferbloat** y los AQMs son de lo más relevante que se agregó después de 2011 en este tema. Linux usa **fq_codel** por defecto desde el kernel 3.5 (2012). RFC 9000 es **BBR** (draft→ estándar informativo).

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-03, Problema 1 (30 puntos) — ECN y control de congestión

> **Fuente:** Examen `eirc1103` / `sirc1103` — 21 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf **Este es el problema más importante de toda la materia para el examen actual.**

Enunciado (texto del examen)

> ECN (Explicit Congestion Notification) es un protocolo que actúa en capa de red y capa de transporte que permite que los hosts de una sesión TCP reciban notificaciones explícitas de congestión generadas por los routers de la red que reenvían los paquetes de dicha sesión. Un paquete IP puede estar marcado como: NO_ECN, ECN_AWARE o ECN_CONG.
>
> TCP puede activar ECN solicitando a su entidad de red marcar la bandera ECN_AWARE del encabezado IP de todos los paquetes que trasportan segmentos de dicha sesión. En caso de congestión, los routers deben marcar la bandera ECN_CONG del encabezado IP en todos aquellos paquetes marcados con ECN_AWARE antes de reenviarlos. Cuando el receptor recibe paquetes marcados con la bandera ECN_CONG le notifica al emisor que existe congestión marcando la bandera CONG_DETECTED del encabezado de los segmentos TCP.
>
> Primitivas: - `void setCongWinSize(int valor)` / `int getCongWinSize()` - `void setCONGReceived()` : Marca los próximos segmentos TCP a enviar con la bandera CONG_DETECTED.
>
> Tipos: `TCP_Segment{ int destPort, srcPort, seqNum, ackSeqNum void[] data bool cong_detected IP_Packet prev_header ... } IP_Packet{ ip_addr src, dst enum_ecn ecn_field // {NO_ECN, ECN_AWARE, ECN_CONG} ... }`
>
> **a)** Cuando el emisor recibe un segmento TCP con la bandera CONG_DETECTED marcada, interpreta dicha situación como congestión. ¿Qué eventos interpreta TCP sin ECN como una señal de congestión en la red?
>
> **b)** Considere una implementación de TCP que carga el segmento TCP recibido en una estructura de tipo `TCP_Segment` que se usa como parámetro para invocar la función `int TCP_congDetection(TCP_Segment segmento)` cuya salida es el estado de congestión. Implemente dicha función considerando los mecanismos tradicionales de detección de congestión de TCP y las mejoras provistas por ECN (si se invoca `TCP_congDetection` con NULL como parámetro, es porque existió un timeout para la recepción de un ACK).
>
> **c)** TCP utiliza la función `TCP_congDetection` y le pasa el resultado a la función `void TCP_congControl(Int congStatus)` que implementa el control de congestión tradicional de TCP y los mecanismos necesarios para utilizar ECN. Implemente dicha función considerando que una vez detectada la congestión se pasa al estado "slow start" (arranque lento).

Parte a) — La respuesta son 4 palabras

Pregunta qué hace TCP sin ECN. La respuesta oficial:

> TCP interpreta que hay congestión en la red cuando se da un evento de pérdida de paquete. Un evento de pérdida puede ser por timeout en la recepción de un ACK o por la recepción de 3 ACKs duplicados.

Por qué esos dos y no otros. Ambos son **evidencias de pérdida**, y la pérdida es la única señal que TCP puede interpretar sin que nadie se lo diga:

Evento	Qué evidencia	Gravedad
Timeout (RTO vencido)	no llegó ni el ACK del siguiente → pérdida **masiva** o ruta caída	alta
3 ACKs duplicados	el siguiente llegó, pero el esperado no → pérdida **aislada**	baja
ACK llega "normal"	nada	ninguna
RTT subió	**cola**, no pérdida	ninguna (TCP clásico no reacciona)

La comparación con ECN (que es lo que la pregunta quiere):

```
SIN ECN: el router DESCARTA → hay pérdida → TCP reacciona a la pérdida
CON ECN: el router MARCA CE → no hay pérdida → TCP recibe CONG_DETECTED
```

O sea: **con ECN, TCP deja de inferir y pasa a serCournotificado**. Esa es la frase para el examen.

> **Tope para 6 puntos:** nombrá los dos eventos (pérdida por timeout, pérdida por 3 ACKs duplicados), explicá que ambos son señales de **pérdida de paquete**, y contrastá con que ECN sustituye esa inferencia por una **notificación explícita** que evita la pérdida.

Parte b) — `TCP_congDetection`

Solución oficial (del PDF):

```
int ultimo_ack, contador = 0;
int TCP_extractCongData(TCP_Segment paquete){
 int estadoCongestion = 0;
 if (paquete == NULL){
 contador = 3;
 }
 else if (paquete.ackSeqNum == ultimo_ack){
 contador++;
 }
 else if (paquete.ackSeqNum > ultimo_ack){
 ultimo_ack = paquete.ackSeqNum;
 contador = 0;
 }

 if (contador == 3) {
 estadoCongestion = 1;
 contador = 0;
 }

 if (paquete != NULL and paquete.CONG_DETECTED) {
 estadoCongestion = 1;
 }
 if (paquete != NULL and paquete.prev_header.ecn_field == ECN_CONG){
 setCONGReceived();
 }
 return estadoCongestion;
}
```

Análisis de la solución oficial, punto por punto. Esto es lo que hay que aprender: no solamente copiar, sino entender qué está bien y qué está raro.

✅ **Lo que está bien:**

1. `if (paquete == NULL) contador = 3;` — **el timeout se traduce a "ya tengo 3 ACKs duplicados"**, es decir, se fuerza el mismo umbral. Es elegante: un solo `if (contador == 3)` más abajo maneja los dos casos. Es la decisión de diseño clave, y por eso la cátedra la premia.
2. `ackSeqNum == ultimo_ack → contador++` y `ackSeqNum > ultimo_ack → ultimo_ack = ...; contador = 0` — el contador de duplicados se **resetea** correctamente cuando llega progreso real.
3. `estadoCongestion = 1` si `paquete.CONG_DETECTED` — **la vía de ECN**.

⚠️ **Lo que está mal o es discutible — y esto también se pregunta:**

1. **`paquete.CONG_DETECTED` en mayúsculas.** En C, los campos que definen el enunciado son `cong_detected`. La solución oficial escribe `paquete.CONG_DETECTED`, que no compila contra el tipo dado. Es un error de transcripción del profesor. **En el examen usá `paquete.cong_detected`**, el nombre del enunciado.
2. **La función se llama `TCP_extractCongData`, no `TCP_congDetection`.** Otro error de la solución: el enunciado pide `TCP_congDetection`.
3. **`contador = 3` en el caso de timeout, seguido de `contador = 0` en el `if`.** Funciona, pero es frágil: si mañana se cambia el umbral a 4, el timeout deja de detectarse. **Mejor forma:** un booleano explícito.
4. **`setCONGReceived()` está en el lugar equivocado.** Este es el punto más interesante. Analicemos:

``` Solución oficial: el EMISOR, dentro de TCP_congDetection, ve que el paquete recibido trae ecn_field == ECN_CONG, y llama a setCONGReceived().

Lo que dice el enunciado: "Cuando el RECEPTOR recibe paquetes marcados con la bandera ECN_CONG le notifica al EMISOR ... marcando la bandera CONG_DETECTED" ```

O sea, `setCONGReceived()` **debe llamarlo el receptor**, y su efecto es poner `CONG_DETECTED` en los segmentos **que el receptor envía**. La solución oficial lo puso del lado del emisor, lo cual es conceptualmente incorrecto.

**Pero** hay una lectura que la justifica: si la implementación es **simétrica** (el mismo código para ambos extremos, que es como se escriben los TCP reales), entonces `TCP_congDetection` corre en los dos hosts. En el receptor, el "paquete recibido" trae `ecn_field == ECN_CONG`, y ahí sí tiene sentido llamar a `setCONGReceived()` para avisarle al otro. **Esa es probablemente la intención de la cátedra** — y por eso el enunciado insiste en que la función esté definida para ambos extremos.

5. **Falta `paquete == NULL` en el segundo `if`.** Está, está bien. OK.
6. **Falta manejar el caso `ackSeqNum < ultimo_ack`.** Es ACK viejo o duplicado de un ACK viejo. La solución oficial no lo tiene: con `contador` sin resetear, un ACK viejo podría incrementar el contador falsamente. Es un detalle de robustez.

**Mi solución (corregida, con las dos lecturas):**
```

```
// ---- Estado que persiste entre llamadas (por conexión TCP) ----
static int ultimo_ack = -1; // último ackSeqNum que_STR saw avanzar
static int dupACK = 0; // ACKs duplicados consecutivos
static bool ECN_soportado = false; // se negoció ECN en el handshake

#define UMBRAL_DUPACK 3

int TCP_congDetection(TCP_Segment seg) // corre en AMBOS extremos
{
 int congStatus = 0; // 0 = todo bien, 1 = congestión

 if (seg == NULL) {
 // --- Evento 1: TIMEOUT. No llegó el ACK a tiempo.
 // Es la señal MÁS grave: o se perdió todo, o la ruta se cayó.
 dupACK = UMBRAL_DUPACK; // lo forzamos al umbral
 dupACK = 0; // (lo consume el reset de abajo)
 congStatus = 1;
 }
 else {
 // --- Evento 2: ACKs duplicados -> fast retransmit
 if (seg->ackSeqNum > ultimo_ack) {
 // hubo PROGRESO: hay que resetear el contador
 ultimo_ack = seg->ackSeqNum;
 dupACK = 0;
 }
 else if (seg->ackSeqNum == ultimo_ack) {
 dupACK++; // ACK repetido
 }
 // si ackSeqNum < ultimo_ack: ACK obsoleto, lo ignoramos (no cuenta)

 if (dupACK >= UMBRAL_DUPACK) {
 congStatus = 1; // fast retransmit
 dupACK = 0; // ya se decidió, se rearma
 }

 // --- Evento 3: ECN, visto POR EL RECEPTOR.
 // El router marcó CE (ECN_CONG). Avisamos al emisor putting the flag
 // CONG_DETECTED en lo que mandemos de aquí en más.
 if (seg->prev_header.ecn_field == ECN_CONG) {
 setCONGReceived(); // marca los próximos segmentos
 }
 }

 // --- Evento 4: ECN, visto POR EL EMISOR.
 // El receptor ya nos puso CONG_DETECTED: congestión explícita, sin pérdida.
 if (seg != NULL && seg->cong_detected) {
 congStatus = 1;
 }

 return congStatus;
}
```

Qué cambió respecto de la solución oficial y por qué:

Cambio	Por qué
`&&` en vez de `and`	`and` es una extensión de ISO C, no C estándar. En un examen de papel no importa, pero en código que compila, sí.
`cong_detected` en minúsculas	Es el nombre del campo en el tipo del enunciado.
`>=` en vez de `==`	Si llegan 4 o 5 duplicados seguidos, con `==` te lo perdés. Con `>=` no.
Timeout no usa el truco de `dupACK=3`	Es menos frágil: no depende del valor del umbral. La versión oficial funciona, la mía no se rompe si cambiás el umbral.
`ECN_CONG` → `setCONGReceived()` **solo en el** `else`	Evita un NULL-dereference: con `seg == NULL` no hay `prev_header`. La solución oficial lo protege con `paquete != NULL and`, que también funciona, pero es más fácil de leer así.
ACK obsoleto ignorado	Robustez.
Comentarios con el **por qué** de cada rama	Esto es lo que suma puntos.

> **Tope para 12 puntos:** tenés que tener los **dos eventos de pérdida** (timeout y 3 dupACKs), el **reset del contador** cuando hay progreso, y la **vía de ECN por** `cong_detected`. Todo eso con los comentarios explicando el razonamiento.

Parte c) — `TCP_congControl`

Solución oficial:

```
#define MSS_BYTES ...
int estado = SLOW_START;
int umbral = valor_inicial;
int ultimoTamanoCongWin;

void TCP_congControl(int congStatus)
{
 if (congStatus = 1) { // <-- BUG: "=" en vez de "=="
 estado = SLOW_START;
 umbral = getCongWinSize() / 2;
 setCongWinSize(MSS_BYTES);
 } else {
 if (estado == SLOW_START) {
 setCongWinSize(MSS_BYTES + getCongWinSize());
 if (getCongWinSize() > umbral) {
 estado = CONG_AVOIDANCE;
 }
 } else {
 setCongWinSize(getCongWinSize() + MSS_BYTES*(MSS_BYTES/getCongWinSize()));
 }
 }
}
```

Análisis de la solución oficial:

1. 🔴 **Bug de sintaxis:** `if (congStatus = 1)`**.** Es **asignación**, no comparación. En C esto asigna 1 a `congStatus` y evalúa el resultado, que siempre es verdadero. O sea, **la función SIEMPRE entra en el**

branch de congestión y nunca crece la ventana. Es un error real en la solución publicada. **En el examen escribí `==`.** Si el corrector es una persona, el error no te resta: demuestra que entendiste. Si además lo señalás, mejor.

2. 🟡 **No hay `else` para la ventana mínima.** Si `cwnd` cae por debajo de 1 MSS (no debería pasar, pero), no hay piso.
3. 🟡 **Falta el `ultimoTamanoCongWin`.** Está declarado pero nunca se usa. En un TCP real lo usarías para detectar si la ventana **creció** de golpe entre dos losses (indicando que la red `no` está congestionada). Es un refinamiento, no lo exige el enunciado.
4. 🟢 **La lógica de `umbral = cwnd/2` y `cwnd = MSS` es correcta** para lo que pide el enunciado (que toda congestión arranque slow start).

Mi solución:

```
#define MSS_BYTES 1460 // MSS típico en Ethernet; el enunciado no lo da

// ---- Estado persistente ----
static int estado = SLOW_START; // SLOW_START | CONG_AVOIDANCE
static int umbral = valor_inicial; // ssthresh: arranca alto (ej. 64 MSS)

// ---------- (c) Control de congestión ----------
void TCP_congControl(int congStatus)
{
 if (congStatus == 1) {
 // ======== CONGESTIÓN detectada ========
 // Guardamos la mitad de la ventana ACTUAL como nuevo umbral:
 // es el punto donde la red empezó a fallar, así que es el punto
 // al que no conviene pasar.
 umbral = getCongWinSize() / 2;

 // El enunciado pide ir a slow start en ambos casos.
 // En producción, con fast retransmit (3 dupACKs) se haría
 // cwnd = umbral en vez de 1 MSS -- es la diferencia Tahoe vs Reno.
 setCongWinSize(MSS_BYTES);
 estado = SLOW_START;
 }
 else {
 // ======== sin congestión: CRECEMOS la ventana ========
 if (estado == SLOW_START) {
 // Por cada ACK recibido sumamos 1 MSS -> se duplica por RTT.
 setCongWinSize(getCongWinSize() + MSS_BYTES);

 // Si superamos el umbral, el crecimiento pasa a ser lineal.
 if (getCongWinSize() >= umbral) {
 estado = CONG_AVOIDANCE;
 }
 }
 else { // CONG_AVOIDANCE
 // +1 MSS por RTT, repartido entre los cwnd/MSS ACKs del RTT:
 // (MSS) / (cwnd/MSS) = MSS²/cwnd por ACK
 // y a lo largo del RTT: (cwnd/MSS)·(MSS²/cwnd) = MSS. Cuadra.
 setCongWinSize(getCongWinSize()
 + MSS_BYTES * (MSS_BYTES / getCongWinSize()));
 }
 }
}
```

Tabla de verificación — corrELA a mano para estar seguro:

```
MSS = 1000, umbral inicial = 64000, cwnd = 1000, estado = SLOW_START

RTT 1: sin cong, SLOW_START → cwnd = 1000 + 1000 = 2000
2000 < 64000 → sigue SLOW_START
RTT 2: → 4000 (< 64000)
RTT 3: → 8000
RTT 4: → 16000
RTT 5: → 32000
RTT 6: → 64000 >= 64000 → estado = CONG_AVOIDANCE
RTT 7: cwnd = 64000 + 1000·(1000/64000) = 64000 + 1000·0
⚠️ con división entera esto da 0. El crecimiento real sería
sumar el resto. En la práctica se usa aritmética de punto
flotante o se acumula el residuo.
→ 64016 aprox.
```

> ⚠️ **Sobre la división entera:** `MSS/cwnd` en C entero es 0 cuando `cwnd > MSS`. La solución oficial tiene el mismo problema. Es un bug real que en un examen no te resta, pero **mencionarlo suma**: dice que entendiste el algoritmo. La forma correcta es `cwnd += MSS*MSS/cwnd` con aritmética de punto flotante, o `cwnd += (MSS*MSS)/cwnd` con `long` suficientemente grande.
>
> **Tope para 12 puntos:** los tres casos — congestión (umbral = cwnd/2, cwnd = MSS, slow start), slow start (cwnd += MSS por ACK), congestion avoidance (cwnd += MSS^2/cwnd) — y la transición de régimen al cruzar el umbral.

3.2 Examen 2008-07, Pregunta 2 (10 puntos) — los cuatro algoritmos

> **Enunciado:** Describa el mecanismo de control de congestión de TCP partiendo de explicar los principios básicos de operación de al menos dos de los cuatro algoritmos básicos involucrados.

Los cuatro algoritmos (esto es lo que hay que saber):

#	Algoritmo	Cuándo	Qué hace con `cwnd`
1	**Slow start**	`cwnd < ssthresh`	`cwnd += MSS` por ACK → **duplica por RTT**
2	**Congestion avoidance**	`cwnd ≥ ssthresh`	`cwnd += MSS²/cwnd` por ACK → **+1 MSS por RTT**
3	**Fast retransmit**	3 ACKs duplicados	retransmite **sin esperar** el RTO
4	**Fast recovery**	3 ACKs duplicados	`cwnd = ssthresh = cwnd/2` (no a 1 MSS)

Respuesta modelo (10 puntos). Ver §1.4, que lo desarrolla entero. Los puntos están en: (a) nombrar los cuatro, (b) decir **qué variable** decide el slow start vs. congestion avoidance, (c) decir **qué tres cosas** pasan con 3 dupACKs, (d) distinguir Tahoe de Reno.

3.3 Examen 2009-07, Pregunta 1 (10 puntos)

> **Enunciado:** a) En un protocolo de capa de transporte: ¿cuál es la diferencia entre Control de Flujo y Control de Congestión? b) ¿Cómo hace TCP para detectar la existencia de congestión en la red?

a) flujo = el receptor no puede absorber; congestión = la red no puede transportar. Se respondé con la tabla de §1.1. **El punto clave para el profesor:** los dos se resuelven reduciendo la ventana efectiva, pero por **motivos distintos**, y confundirlos es el error típico.

b) Las dos señales: pérdida por **timeout** y pérdida por **3 ACKs duplicados**. Sumá que con ECN hay una tercera: la **marca CE** del router.

3.4 Examen 2009-12, Pregunta 2 (10 puntos)

> **Enunciado:** Se considera un servicio de transmisión confiable. a) Señale las diferencias de implementación en capa de transporte vs. capa de enlace. b) En una red de paquetes arbitraria, ¿es posible implementar un servicio de transmisión confiable en capa de enlace? Justifique.

a) En la capa de enlace la fiabilidad es **por salto** y **por enlace**, sin estado entre saltos, sin número de secuencia global, y los buffers son de un solo paquete. En la capa de transporte la fiabilidad es **extremo a extremo**, con números de secuencia de bytes, ACK acumulativos, control de congestión, y funciona **aunque los saltos intermedios no sean fiables**.

b) No es confiable — y esto es lo que el enunciado quiere que argumentes:

```
Si la capa 3 (IP) descarta paquetes, la capa 2 no lo sabe:
no hay número de secuencia que atraviese los routers,
no hay ACK que vuelva,
no hay timeout que pueda distinguir "se perdió" de "tardó".

Peor: la retransmisión de capa 2 en el último salto
puede NO arreglar nada, porque el paquete se perdió en el router de al lado.

Ejemplo mínimo que lo demuestra:
emisor --(1)--> router A --(2)--> router B --(3)--> receptor
Se pierde en (2). La capa 2 de (1)-(2) no ve nada raro.
La capa 2 de (2)-(3) ve llegar bien y jamás retransmite.
Resultado: paquete perdido para siempre, sin detección.
```

Conclusión para el examen: la fiabilidad **extremo a extremo** es la única que funciona sobre una red no confiable. Por eso está **arriba** y no abajo.

PARTE 4 — CIERRE

📌 Resumen

1. **Control de flujo** = el receptor no puede absorber → lo controla `rwnd`.
2. **Control de congestión** = la red no puede transportar → lo controla `cwnd`.
3. La ventana efectiva es **`min(cwnd, rwnd)`** — un mínimo porque ambas son cotas superiores.
4. **Slow start**: `cwnd += MSS` por ACK → **duplica por RTT** (exponencial base 2).
5. **Congestion avoidance**: `cwnd += MSS²/cwnd` por ACK → **+1 MSS por RTT** (lineal).
6. El régimen lo decide **`cwnd` vs. `ssthresh`**.
7. **3 ACKs duplicados** → fast retransmit (porque 3 hace improbable el reordenamiento).
8. **Timeout** → pérdida grave → slow start desde 1 MSS.
9. **Reno** distingue: 3 dupACKs → `cwnd/2` (fast recovery); timeout → 1 MSS.
10. **ECN**: el router marca **CE** en vez de descartar; el receptor avisa con **CONG_DETECTED**; **cero paquetes perdidos**.

✅ Autoevaluación

1. ¿Por qué la ventana efectiva es el **mínimo** de `cwnd` y `rwnd` y no la suma?

▶ Respuesta

2. En slow start, ¿por qué `cwnd` se duplica en cada RTT? Explicá desde el número de ACKs.

▶ Respuesta

3. ¿Por qué exactamente 3 ACKs duplicados y no 1?

▶ Respuesta

4. ¿Qué valor toma `cwnd` con 3 dupACKs en Tahoe? ¿Y en Reno? ¿Y con timeout en ambos?

▶ Respuesta

5. ¿Qué cambia en la red con ECN que no cambia con la detección por pérdida?

▶ Respuesta

6. Escribí la fórmula de crecimiento en congestion avoidance y justificá por qué da +1 MSS por RTT.

▶ Respuesta

7. Un cliente tiene `cwnd = 8 KB` y el receptor anuncia `rwnd = 2 KB`. (a) ¿Cuántos bytes manda por RTT? (b) Si el RTT es 40 ms, ¿cuál es el throughput?

▶ Respuesta

8. En el problema de 2011-03, ¿por qué el caso `seg == NULL` (timeout) se implementa poniendo `dupACK = 3`?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1103` / `sirc1103`** — Problema 1 completo (30 pts, ECN), P2 (10 pts, control de congestión)
- **Examen `eirc0807` / `sirc0807`** — P2: los 4 algoritmos (10 pts)
- **Examen `eirc0907` / `sirc0907`** — P1: flujo vs. congestión (10 pts)
- **Examen `sirc0912`** — P2: fiable en transporte vs. enlace (10 pts)
- **Tanenbaum & Wetherall**, *Redes deadoras* 5ª ed., cap. 6 §6.7 "TCP Congestion Control" (pág. ~600 del PDF)
- **RFC 9293** — TCP (2022, *Internet Standard*; consolida y obsolesce RFC 793 + 5 updates)
- **RFC 5681** — TCP Congestion Control (2009): los cuatro algoritmos
- **RFC 3168** — ECN en IP (2001) · **RFC 9768** — AccECN, ECN más preciso
- **RFC 9331** — L4S (2023) · **RFC 8511** — ABE (2018)
- **RFC 9438** — CUBIC (2023, *Standards Track*, obsolesce RFC 8312)
- **RFC 6937** — PRR, *Experimental*
- **RFC 6928** — Initial Window = 10 MSS (2013) · **RFC 7414** — roadmap de las RFCs de TCP
- `draft-ietf-ccwg-bbr` — BBRv3 (*Experimental*, nunca fue RFC)
- **Clase siguiente:** 22 — Taller TCP: diagramas de intercambio resueltos